

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3: Разработка одностраничного
веб-приложения (SPA) с использованием фреймворка
Vue.JS

Выполнил:

Емельков Матвей

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Каким требованиям ваше приложение должно соответствовать:

- 1) Должен быть подключён роутер
- 2) Должна быть реализована работа с внешним API (желательно посредством axios)
- 3) Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- 4) Использование composables для выделения повторяющегося функционала в отдельные файлы

Ход работы

Для начала я инициализировал проект на основе Vite с поддержкой Vue 3 и установил необходимые зависимости: vue, vue-router, axios и bootstrap

Затем я настроил Vue Router с защищенными маршрутами: неавторизованный пользователь автоматически перенаправляется на страницу входа, а уже вошедший - с /login на главную. Использую createWebHashHistory, чтобы SPA корректно работало без настройки сервера

Далее я вынес повторяющийся функционал в три composable-функции:

- useAuth.js - реактивное состояние текущего пользователя, функции login и logout с синхронизацией localStorage. Состояние объявлено на уровне модуля, поэтому все компоненты разделяют один экземпляр
- useTheme.js - переключение тёмной/светлой темы через CSS-переменные и localStorage
- useApi.js - централизованный экземпляр axios с базовым URL http://localhost:3000. Все HTTP-методы (для экспериментов, моделей, пользователей) возвращаются из одной функции
- useLocale.js - реализует механизм интернационализации интерфейса. Текущий язык хранится в модульном реактивном ref на уровне модуля, что гарантирует единое состояние локали для всех

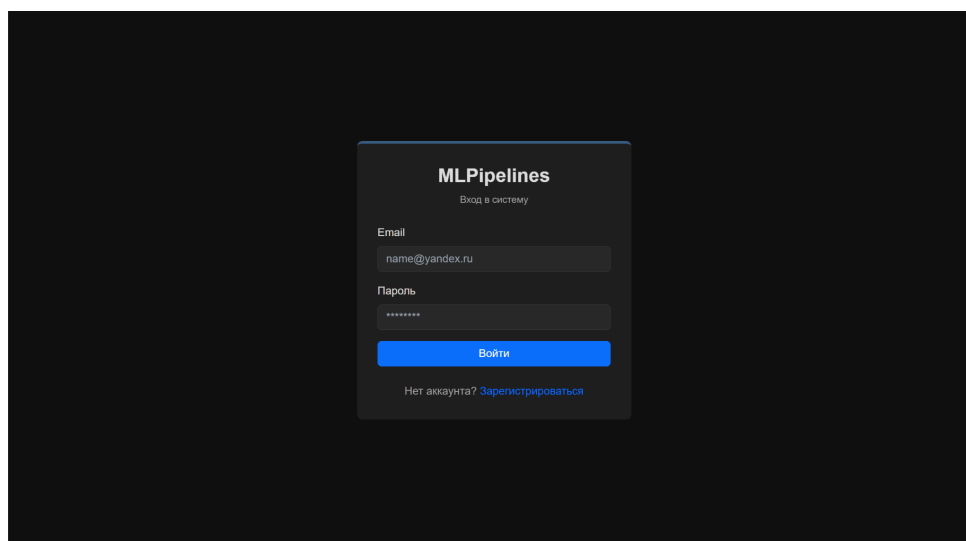
компонентов приложения без использования глобального хранилища. При инициализации значение языка восстанавливается из `localStorage`, что позволяет сохранять выбор пользователя между сессиями

После этого я разбил приложение на компоненты. Структура `src/`:

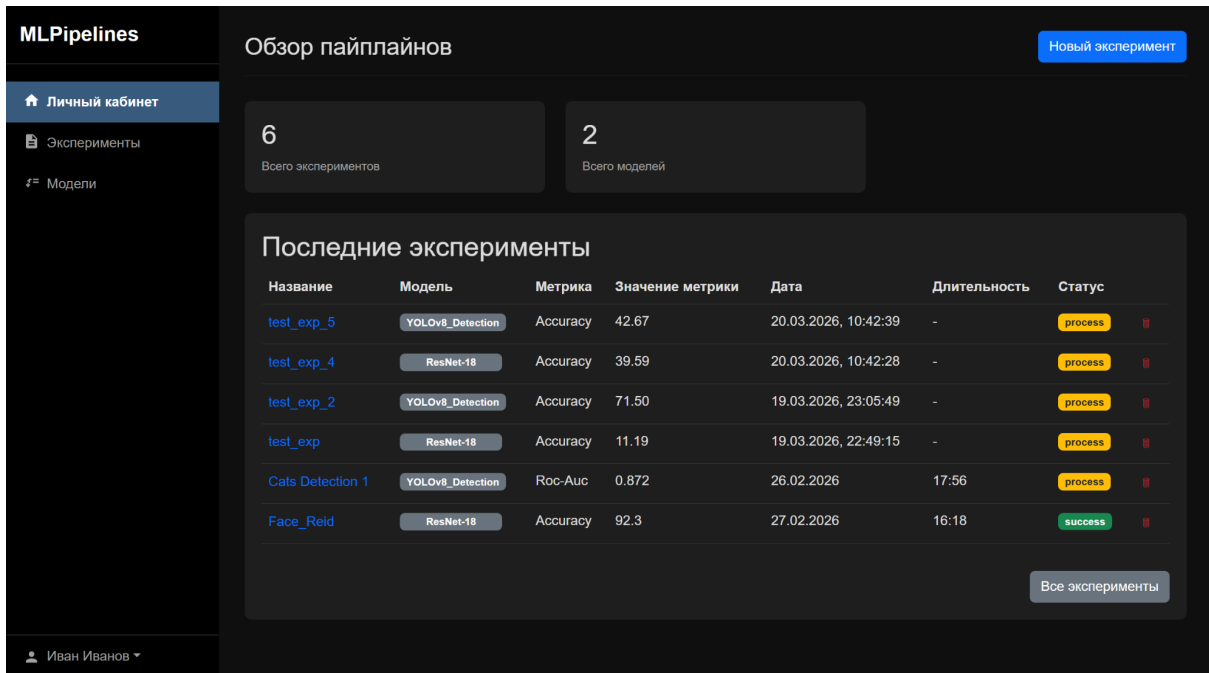
```
src/
├── composables/ useAuth.js, useTheme.js, useApi.js, useLocale.js
├── components/  AppLayout.vue, AppSidebar.vue,
                SettingsModal.vue, NewExperimentModal.vue,
                NewModelModal.vue
├── views/       LoginView.vue, RegisterView.vue,
                DashboardView.vue, ExperimentsView.vue,
                ExperimentDetailsView.vue, ModelsView.vue
├── router/      index.js
└── main.js
```

`AppLayout.vue` — общая обёртка для всех авторизованных страниц: содержит `AppSidebar` и `SettingsModal`, управляет их состоянием через `ref`. `AppSidebar.vue` использует `router-link` с `exact-active-class` для корректной подсветки активного пункта меню

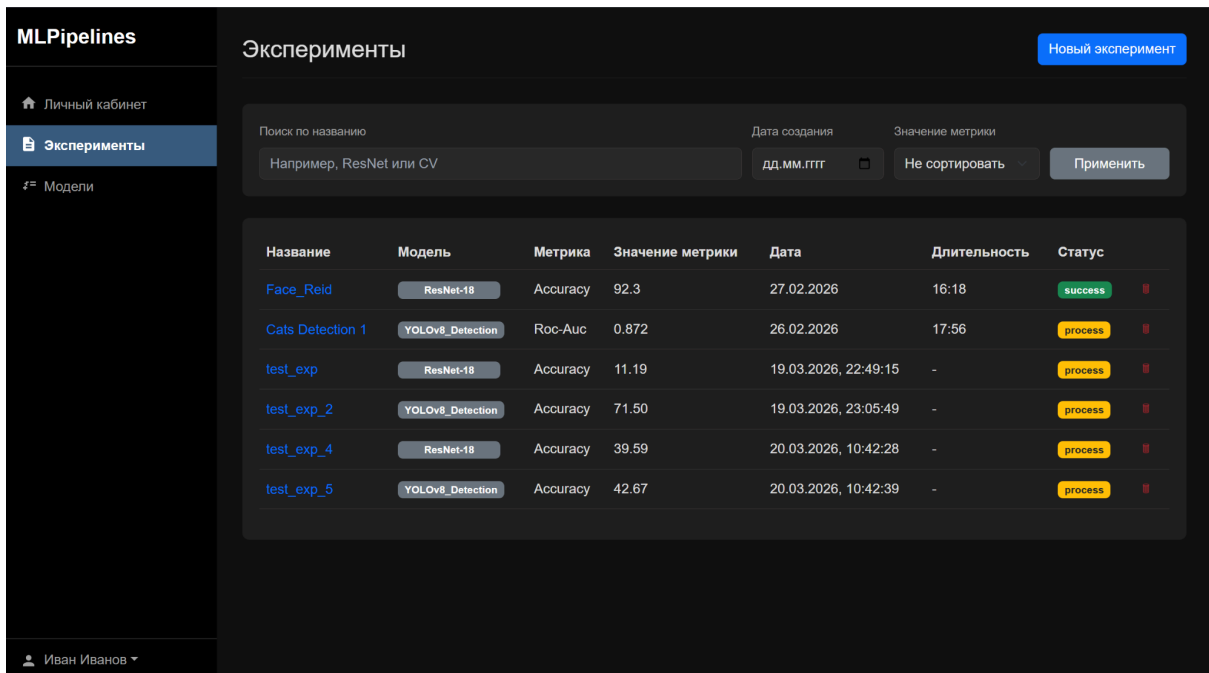
Страницы авторизации реализованы как отдельные `views`. При попытке входа компонент обращается к `json-server` через `useApi`, находит пользователя по `email` и проверяет пароль. В случае ошибки отображается `inline`-сообщение в форме - без `alert`

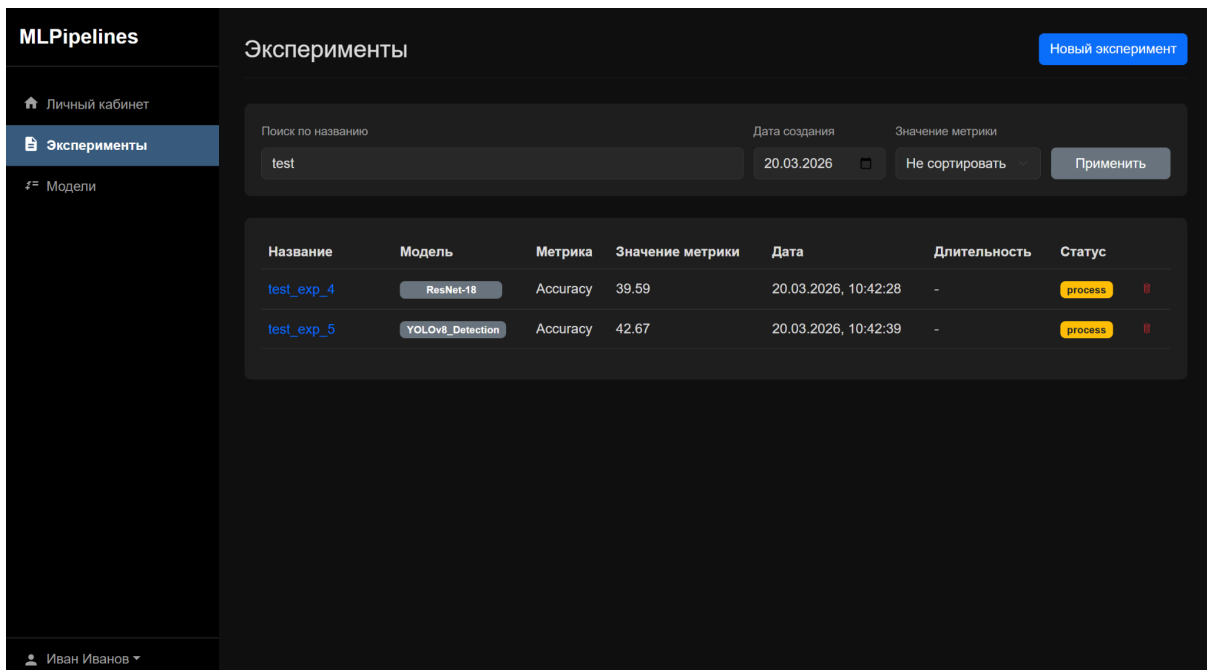


Дашборд подгружает данные через Promise.all при монтировании компонента и отображает последние 6 экспериментов через computed

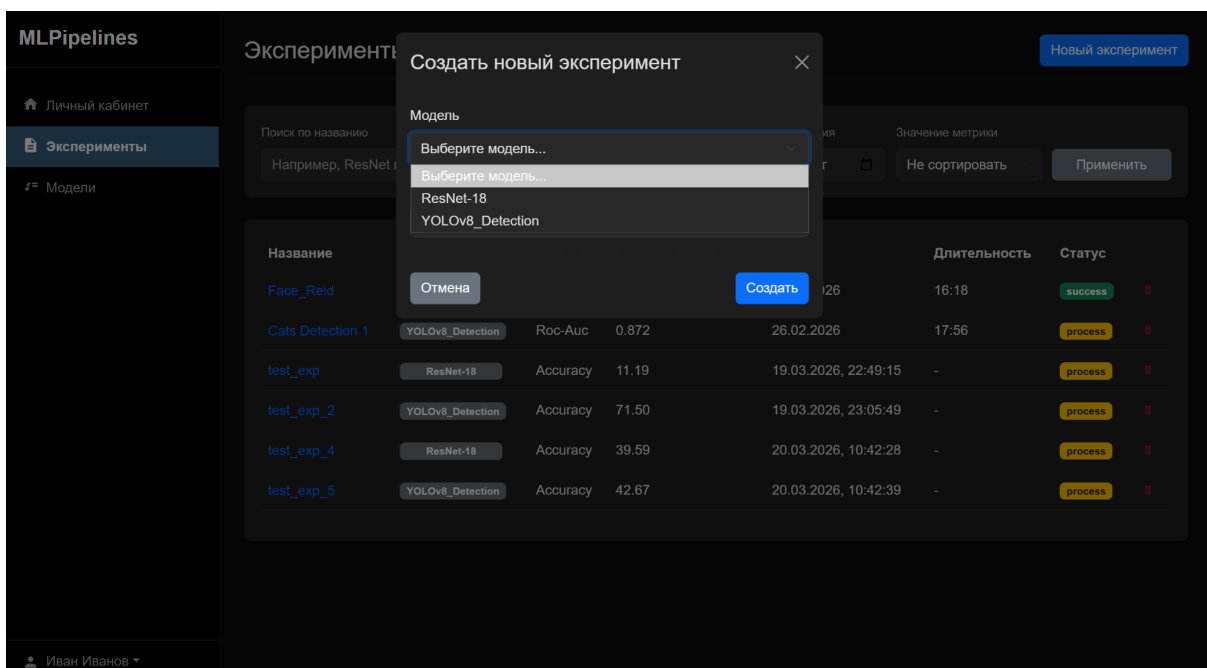


На странице экспериментов реализована фильтрация: при нажатии «Применить» делается запрос к json-server с фильтром по userId, после чего результат дополнительно фильтруется на клиенте по названию и сортируется по значению метрики через parseFloat - это обеспечивает корректную числовую сортировку при неоднородных данных

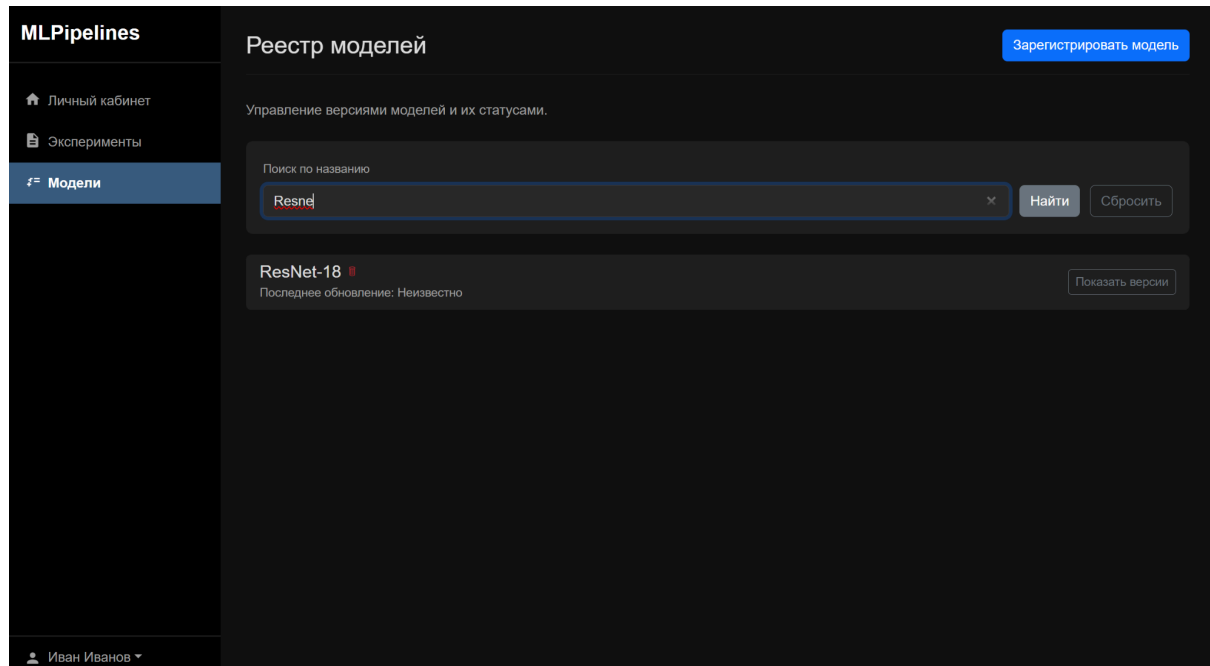




Модальное окно создания эксперимента реализовано через `<Teleport to="body">` и `v-if` без Bootstrap JS. Модель выбирается из выпадающего списка существующих моделей пользователя. При выборе модели название эксперимента автоматически заполняется по шаблону `{модель}_{дата}`, но остаётся редактируемым. Валидация выполняется до закрытия окна



На странице моделей реализован поиск по названию: при каждом запросе делается новый вызов к API с фильтрацией на клиенте. Версии модели (эксперименты с совпадающим полем model) раскрываются по клику без перезапроса к серверу



Вывод

В процессе выполнения работы я мигрировал приложение с ванильного js на Vue 3. Я разбил интерфейс на переиспользуемые компоненты и вынес повторяющуюся логику (авторизация, тема, API-запросы) в composable-функции. Настройка Vue Router с навигационными guard-ами позволила реализовать защищённые маршруты. Работа с axios через централизованный composable упростила обращение к json-server и сделала код более поддерживаемым